SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	**Details**	**Field**	**Details**
Student Name		Register Number	
Department / Section		Academic Year / Semester	
Faculty Name		Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	**Title**	**Max**	**Awarded**	**Evidence / evaluation focus**
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations

5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks
8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing

9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	P	HTTP 404 test case documented; actual execution record required
Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

OVERALL REMARKS

_____	_____	_____
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	**Evidence expected**	**Status**	**Evaluator note**
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance

4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	**Status**	**Observation**
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required
Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented

Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and online-exam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university

- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet
- Comparison between desktop software, client–server software and web-delivered software
- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization

- Propose which applications should run as virtual machines and which as containers, with reasons
- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)
- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model
- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lock-in
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multi-zone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort
- Estimate a simple 3-year cost comparison using stated assumptions
- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	**Title**	**Bloom's Level**	**Marks**
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3

Task	Title	Bloom's Level	Marks
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50

ANSWERS:

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

1.1 Definition of Cloud Computing

Cloud computing is the delivery of computing services such as servers, storage, databases, networking and software through the internet.

It allows users to access computing resources whenever required without purchasing and maintaining physical hardware.

Examples: AWS, Microsoft Azure and Google Cloud.

1.2 Five Essential Characteristics of Cloud Computing

1. On-Demand Self-Service: Users can access computing resources whenever needed without contacting the service provider.

2. Broad Network Access: Cloud services can be accessed through laptops, mobile phones and tablets using the internet.

3. Resource Pooling: Computing resources are shared among multiple users using virtualization.

4. Rapid Elasticity: Resources can automatically increase or decrease according to demand.

5. Measured Service: Users pay according to the resources they consume.

1.3 Cloud Deployment Models

Model	Explanation	Example
Public Cloud	Services are provided over the internet to multiple customers.	AWS
Private Cloud	Cloud infrastructure is dedicated to one organization.	University private cloud
Hybrid Cloud	Combination of public and private clouds.	Private database and public cloud
Community Cloud	Shared by organizations with similar requirements.	Educational institutions

Recommended Model: Hybrid Cloud

The university should use a hybrid cloud model.

- Sensitive student records can be stored in a private environment.
- Web applications and mobile services can run on the public cloud.
- Backup data can be stored in cloud storage.
- It provides better security, flexibility and scalability.

1.4 Three Problems in the Current System and Cloud Solutions

Current Problem	Cloud Solution
Servers remain idle most of the year.	Resource pooling improves resource utilization.
Servers fail during admissions and exams.	Rapid elasticity provides additional resources during peak periods.
Purchasing servers takes 6–8 weeks.	On-demand self-service provides resources immediately.

1.5 Benefits and Limitations

Benefits:

- Reduces hardware purchasing costs.

- Provides automatic scalability.
- Improves data backup and recovery.
- Allows access from all four campuses.
- Reduces maintenance work.

Limitations:

- Requires a stable internet connection.
- Security and privacy must be managed carefully.
- Cloud services may become expensive with high usage.
- Dependence on cloud service providers.

Conclusion: Hybrid cloud computing is suitable for the university because it provides scalability, security, flexibility and cost savings.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

2.1 Evolution of Computing Infrastructure

Computing infrastructure developed through several stages.

Stage	Explanation
Mainframe Computing	Large central computers served many users through terminals.
Minicomputers	Smaller and less expensive computers were introduced.
Personal Computers	Individual users could perform computing tasks independently.
Client–Server Computing	Clients requested services from centralized servers.
Cluster Computing	Multiple computers worked together as one system.
Grid Computing	Computers in different locations shared processing resources.
Utility Computing	Computing resources were provided based on usage and payment.
Cloud Computing	Computing resources became available over the internet on demand.

2.2 Hardware Evolution

The following hardware improvements made cloud computing possible.

Hardware Development	Importance
Multi-core processors	Perform multiple tasks simultaneously.
Commodity servers	Provide affordable computing power.
High-capacity storage	Store large amounts of data.

High-speed networking	Transfer data quickly between systems.
Large data centres	Support thousands of servers in one location.

2.3 How Each Stage Improved Computing

- Mainframes provided centralized processing.
- Minicomputers reduced the cost of computing.
- PCs provided individual computing facilities.
- Client–server systems enabled resource sharing.
- Clusters improved processing performance.
- Grid computing allowed distributed resource sharing.
- Utility computing introduced pay-per-use.
- Cloud computing combined these advantages with internet-based access and elasticity.

2.4 Current University System

The university currently uses a traditional client–server computing model.

Reasons:

1. It has 12 dedicated physical servers.
2. Applications run on separate servers.
3. Resources are not efficiently shared.
4. Server maintenance is performed locally.
5. There is no disaster recovery facility.

Conclusion: The university should move towards cloud computing to improve resource utilization, scalability and availability.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

3.1 Evolution of Internet Software

Internet software developed through the following stages.

Stage	**Explanation**
Static Web Pages	Display fixed information using HTML.
Dynamic Web	Web pages display changing information using databases and server-side programs.
Web 2.0	Supports user interaction, collaboration and social networking.

Service-Oriented Architecture (SOA)	Applications communicate through reusable services.
Mashups	Combine data and services from multiple sources.
Cloud-Hosted Applications	Applications are hosted on cloud servers and accessed through the internet.

3.2 Role of Important Technologies

1. Browsers

Browsers such as Chrome and Firefox allow users to access web applications.

2. HTTP

HTTP is a communication protocol used to transfer information between clients and web servers.

3. AJAX

AJAX allows web pages to update information without reloading the entire page.

4. APIs

APIs allow different applications to communicate with each other.

Example: The university mobile application can use an API to display student examination results.

3.3 Comparison of Software Types

Feature	Desktop Software	Client–Server Software	Web-Based Software
Installation	Required	Client installation may be required	Usually not required
Access	Individual computer	Connected client	Browser or mobile app
Maintenance	Individual system	Client and server	Mainly centralized
Scalability	Limited	Moderate	High
Internet	Not always required	Usually network required	Usually internet required

3.4 Effect on University Applications

The university should convert its monolithic applications into web-based applications.

Examples:

- Students can view marks through a browser.
- Teachers can update attendance online.
- Students can search library books.

- Parents can check academic progress.
- Mobile applications can access university services through APIs.

Conclusion: Modern internet technologies make it possible to provide university services through browsers, mobile applications and APIs.

TASK 4 – Apply Server Virtualization (5 Marks)

4.1 Definition of Server Virtualization

Server virtualization is a technology that divides one physical server into multiple virtual machines.

Each virtual machine works like an independent computer with its own operating system, memory, CPU and applications.

It helps organizations use hardware resources efficiently.

4.2 Working of a Hypervisor

A hypervisor is software that creates and manages virtual machines.

It distributes the physical server's CPU, RAM and storage among multiple virtual machines.

4.3 Types of Hypervisors

Type	Explanation	Examples
Type-1	Runs directly on physical hardware.	VMware ESXi, Microsoft Hyper-V
Type-2	Runs on top of an existing operating system.	VirtualBox, VMware Workstation

Type-1 is recommended for the university because it provides better performance, security and management for production workloads.

4.4 Types of Virtualization

Type	Explanation	Example
Full Virtualization	Complete hardware environment is provided to a virtual machine.	VMware
Para-Virtualization	Guest operating system is aware of the hypervisor and cooperates with it.	Xen PV
OS-Level Virtualization	Multiple isolated containers share the same operating system kernel.	Docker
Hardware-Assisted Virtualization	Uses processor features to improve virtualization.	Intel VT-x, AMD-V

4.5 Proposed Virtualization Plan

The university has five major applications.

Application	Proposed Technology	Reason
Student Information System	Virtual Machine	Supports legacy applications.
Learning Management System	Containers	Easy scaling during online classes.
Library System	Virtual Machine	Supports existing database software.
Exam Portal	Containers	Handles high traffic during exams.
Email System	SaaS	Reduces maintenance and administration.

4.6 Server Consolidation Calculation

Given:

- Total physical servers = 12
- Average CPU utilization = 15%
- Target host utilization = 70%

Step 1: Calculate total CPU utilization

12 × 15 = 180%

Total equivalent server capacity required = 180%.

Step 2: Calculate the number of physical hosts

Number of hosts = Total utilization ÷ Target utilization

= 180 ÷ 70

= 2.57

Therefore, round up to the next whole number.

Final Answer: 3 Physical Hosts

Explanation: Three physical hosts can support the combined workload while keeping average CPU utilization around 60%, assuming equal server capacity and compatible workloads.

For production, additional capacity should be reserved for failures and unexpected demand. CPU, RAM, storage and network requirements must also be checked.

4.7 Benefits of Virtualization

1. Live Migration: Moves running virtual machines between physical servers with minimal interruption.
2. Snapshots: Saves the current state of a virtual machine for recovery.

3. Resource Over-Commitment: Allows virtual machines to share more allocated resources than the physical server has, subject to actual usage.

4.8 Risks of Virtualization

- Noisy Neighbour: One virtual machine may consume too many resources and affect other machines.
- Hypervisor Failure: A hypervisor failure may affect multiple virtual machines.
- Security Isolation: Incorrect configurations may allow unauthorized access between virtual machines.

Conclusion: Virtualization can reduce the university's 12 physical servers to approximately three hosts under the stated assumptions. It improves utilization, flexibility and management.

TASK 5 – Apply Web Services (4 Marks)

5.1 Definition of Web Services

Web services are software components that allow different applications to communicate over a network using standard protocols.

They help applications exchange data and perform operations.

Example: A mobile application accessing university examination results.

5.2 Types of Web Service Technologies

Technology	Explanation
SOAP	XML-based protocol used for structured communication.
WSDL	Describes the operations and structure of a SOAP web service.
UDDI	Directory used to publish and discover web services.
REST	Architectural style that uses HTTP methods to access resources.

5.3 HTTP Methods

- GET: Retrieve information.
- POST: Create new information or perform an operation.
- PUT: Update existing information.
- DELETE: Remove information.

REST commonly uses JSON, while SOAP uses XML.

5.4 University Web Services

The university should provide the following web services.

1. Student Result Lookup

- Endpoint: /api/students/{id}/results

- Method: GET
- Purpose: Retrieve student examination results.
- Response: Student ID, name, semester, result and percentage.

Recommended: REST because it provides simple and fast access to student results.

2. Fee Payment Service

- Endpoint: /api/fees/payment
- Method: POST
- Purpose: Process student fee payments.
- Request: Student ID, amount and payment method.
- Response: Payment status, transaction ID and amount.

Recommended: REST with secure payment-provider integration, authentication and transaction verification.

3. Library Search Service

- Endpoint: /api/library/books
- Method: GET
- Purpose: Search for available books.
- Response: Book name, author and availability.

Recommended: REST because it supports simple searching and mobile application integration.

5.5 SOAP vs REST

Feature	**SOAP**	**REST**
Format	XML	JSON or XML
Complexity	High	Low
Speed	Generally heavier	Generally lightweight
Security	Supports WS-Security	Commonly uses HTTPS and token-based security
Usage	Enterprise services	Mobile apps and web applications

Conclusion: REST is suitable for most university services because it is lightweight, flexible and easy to integrate with mobile applications. SOAP can be used where strict enterprise messaging standards are required.

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

6.1 Infrastructure as a Service (IaaS)

IaaS provides virtual machines, storage and networking through the internet.

The customer manages the operating system, middleware and applications.

Example: Amazon EC2.

6.2 Platform as a Service (PaaS)

PaaS provides a ready-made platform to develop, test and deploy applications.

The cloud provider manages the infrastructure and operating system.

Example: Google App Engine.

6.3 Software as a Service (SaaS)

SaaS provides complete software applications through the internet.

Users can access the software without installing or maintaining the server infrastructure.

Example: Google Workspace.

6.4 Responsibility Comparison

Layer	IaaS	PaaS	SaaS
Physical Hardware	Provider	Provider	Provider
Networking	Provider	Provider	Provider
Storage	Provider	Provider	Provider
Operating System	Customer	Provider	Provider
Middleware	Customer	Provider	Provider
Runtime	Customer	Provider	Provider
Application	Customer	Customer	Provider
Data and access configuration	Customer	Customer	Shared responsibility

6.5 Categorization of University Applications

Application	Service Model	Reason
Student Information System	IaaS	Existing legacy application requires control over the operating system.
Learning Management System	PaaS	Easy application development and automatic scaling.
Library System	IaaS	Existing database and application can be migrated with fewer changes.

Exam Portal	PaaS	Supports rapid development and scaling during examinations.
Email System	SaaS	Complete email services are available without managing servers.

6.6 Comparison of Service Models

Feature	IaaS	PaaS	SaaS
Control	High	Medium	Low
Cost	Pay for infrastructure	Pay for platform usage	Subscription-based
Scalability	High	Very high	Provider-managed
Maintenance	More	Less	Very little
Vendor Lock-in	Moderate	Higher	Higher

6.7 Reasons for Selecting These Models

Student Information System – IaaS

The existing student information system is a legacy application. IaaS allows the university to migrate it without completely rewriting the application.

Learning Management System – PaaS

PaaS supports application development and automatic scaling. It is useful when many students access learning materials simultaneously.

Library System – IaaS

The library system may depend on existing databases and software. IaaS provides control over its environment.

Exam Portal – PaaS

The examination portal requires high performance during exam periods. PaaS provides automatic scaling and reduces infrastructure maintenance.

Email System – SaaS

A SaaS email service reduces administration, maintenance and hardware costs.

Conclusion: A combination of IaaS, PaaS and SaaS is suitable because every university application has different technical and management requirements.

TASK 7 – Examine XaaS (3 Marks)

7.1 Definition of XaaS

XaaS stands for Anything as a Service.

It is a cloud computing concept in which different IT services are delivered over the internet according to customer requirements.

It reduces the need to purchase and maintain physical infrastructure.

7.2 Types of XaaS

Type	Full Form	Explanation
DaaS	Desktop as a Service	Provides virtual desktops through the internet.
BaaS	Backend as a Service	Provides backend services for applications.
FaaS	Function as a Service	Runs individual functions without managing servers.
DBaaS	Database as a Service	Provides managed database services.
SECaaS	Security as a Service	Provides cloud-based security services.
DRaaS	Disaster Recovery as a Service	Provides backup and disaster recovery facilities.

7.3 Two Recommended XaaS Services

1. Database as a Service (DBaaS)

The university can use DBaaS to store student information, examination results and attendance records.

Benefits:

- Automatic database backups.
- Easy database management.
- High availability.
- Automatic storage scaling.

2. Disaster Recovery as a Service (DRaaS)

The university currently does not have a disaster recovery site.

DRaaS provides backup and recovery services through another cloud location.

Benefits:

- Protects important university data.
- Reduces downtime.
- Supports business continuity.
- Helps recover applications after failures.

7.4 Risks of Multiple XaaS Providers

- Increased management complexity.
- Different security policies.
- Vendor lock-in.

- Additional subscription costs.
- Data transfer difficulties.

Conclusion: DBaaS and DRaaS are highly useful for the university because they improve database management, data protection and business continuity.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

8.1 Proposed Cloud Architecture

The university should adopt a hybrid cloud architecture using virtualization, web services, IaaS, PaaS and SaaS.

8.2 Components of Cloud Architecture

Component	Function
Users	Students, teachers and staff access the system.
Internet and Firewall	Provides connectivity and protects the network.
Load Balancer	Distributes requests among available servers.
Web and Application Tier	Runs university applications.
API Layer	Allows different applications to communicate.
Database Tier	Stores student records, fees, marks and attendance.
Cloud Storage	Stores documents and backup files.
Disaster Recovery Region	Maintains backup services for recovery during failures.

8.3 Components and Service Models

Component	Technology	Service Model
Virtual Servers	Cloud virtual machines	IaaS
Application Platform	Managed application platform	PaaS
Database	Managed database	DBaaS
Email	Cloud email	SaaS
Storage	Cloud object storage	IaaS / Storage-as-a-Service
API Gateway	Managed API service	PaaS
Backup	Cloud backup service	DRaaS

8.4 How Elasticity Works

During admissions and examination periods, many students access the university system simultaneously.

The cloud solution uses auto-scaling.

1. Monitoring tools check CPU, memory and network usage.
2. When traffic increases, additional application instances are created.
3. The load balancer distributes requests among available instances.
4. When traffic decreases, unnecessary instances are removed.
5. This reduces costs during normal periods.

8.5 Security and Availability Measures

- Authentication: Only authorized users can access the system.
- Encryption: Student information is encrypted during transmission and storage.
- Backup: Regular database backups are maintained.
- Multi-Zone Deployment: Applications run across multiple availability zones.
- Firewall: Prevents unauthorized network access.
- Access Control: Students, teachers and administrators receive different permissions.

Conclusion: The proposed hybrid cloud architecture provides scalability, availability, security and centralized management for all university campuses.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

9.1 Aim

To implement a simple cloud-based university service using a local development environment.

9.2 Software Requirements

- Python
- Visual Studio Code
- Flask
- Docker (optional)
- Web browser

9.3 Implementation Steps

1. Create a project folder named UniversityCloud.
2. Create a simple student result management application using Python and Flask.
3. Install the required Flask package.
4. Start the application using the local development server.
5. Open the application in Google Chrome.

6. Access student information through the REST API.
7. Verify the output for different student IDs.

9.4 Testing Results

Test Case	Input	Expected Output
Test 1	Open home page	University Cloud Management System
Test 2	Student ID 101	Student details
Test 3	Student ID 102	Student details
Test 4	Student ID 105	Student not found

9.5 Screenshots to Include

Take screenshots of the following:

1. Application development in VS Code.
2. Required software installation.
3. Running application in the terminal.
4. Student result displayed in Chrome.
5. Docker container running, if Docker is used.

9.6 Issues Faced

- Required software was not installed initially.
- Incorrect URL caused errors.
- Port 5000 may already be in use.
- Docker requires proper configuration.

Conclusion: The proof-of-concept demonstrates how university student information can be accessed through a REST API. The same approach can be extended to cloud platforms.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

10.1 Comparison of Different Solutions

The university can choose between on-premise infrastructure, IaaS and PaaS/SaaS.

Feature	On-Premise	IaaS	PaaS/SaaS
Cost	High initial cost	Pay-as-you-use	Subscription-based
Scalability	Limited	High	Very high
Availability	Depends on local servers	Cloud-based	Provider-managed
Security	University-managed	Shared responsibility	Shared responsibility

Maintenance	High	Medium	Low
Flexibility	Limited	High	Depends on provider

10.2 Capital Expenditure and Operational Expenditure

CapEx (Capital Expenditure):

Money spent on purchasing physical servers, storage devices and networking equipment.

OpEx (Operational Expenditure):

Regular payments for cloud services, software subscriptions, storage and maintenance.

Cloud computing reduces initial investment by converting many infrastructure expenses into operational expenses.

10.3 Three-Year Cost Estimation

The following is an illustrative cost comparison in Indian Rupees. These are assumed planning estimates, not actual provider quotations.

Assumptions:

- 12 existing physical servers.
- Four campuses.
- 25,000 students and 2,000 staff.
- Three-year evaluation period.
- Similar application functionality in all options.

Estimated amounts in ₹ lakh.

Expense	**On-Premise**	**IaaS Cloud**	**PaaS/SaaS Cloud**
Initial setup	107	15	10
Annual operation	18	36	30
Three-year operating cost	54	108	90
Total 3-year cost	161	123	100

Explanation:

- On-premise requires purchasing and maintaining hardware.
- IaaS reduces hardware purchasing but requires server and operating system management.
- PaaS/SaaS reduces infrastructure maintenance and administrative effort.

Therefore, PaaS/SaaS provides the lowest estimated three-year cost under these assumptions.

10.4 Main Risks and Mitigation

Risk	Explanation	Mitigation
Vendor Lock-in	Dependence on one cloud provider	Use open standards and portable containers.
Data Privacy	Unauthorized access to student data	Encryption and access control.
Downtime	Cloud service interruption	Multi-zone deployment and backups.
Skills Gap	Staff may lack cloud knowledge	Provide cloud training.

10.5 Migration Roadmap

The university should migrate applications in a planned manner.

Phase 1 – Planning

Study existing applications, databases, security requirements and workloads.

Phase 2 – Email Migration

Move the email system to SaaS because it is easier to migrate and manage.

Phase 3 – Library Migration

Move the library system to IaaS and validate database operations.

Phase 4 – LMS Migration

Deploy the Learning Management System using PaaS.

Phase 5 – Student Information System

Migrate student records carefully with data validation.

Phase 6 – Exam Portal

Deploy the exam portal with auto-scaling and load balancing.

Phase 7 – Testing and Monitoring

Perform security, performance, backup and disaster recovery tests.

10.6 Final Recommendation

The university should adopt a hybrid cloud model using a combination of IaaS, PaaS and SaaS.

Reasons:

1. Reduces hardware purchasing costs.
2. Provides automatic scalability.
3. Improves availability.
4. Protects sensitive student information.

5. Reduces infrastructure maintenance.
6. Supports all four campuses.

Conclusion: Cloud migration is the best long-term solution for the university because it improves performance, flexibility, cost management and service availability.

TASK 11 – Re-evaluate and Adapt the Design (Day-4 Design Challenge) (8 Marks)

11.1 Impact Analysis (2 Marks)

The university introduces new requirements on Day 4.

Changed Requirements:

1. Launch a new mobile application.
2. Provide student-record services to 40 affiliated colleges.
3. Provide online fee-payment services.
4. Expected traffic increases to 10 times the current peak.
5. Availability must be 99.9%.
6. The solution must operate within a limited budget.

Changes from the Original Requirements

Original Requirement	Changed Requirement
25,000 students and 2,000 staff	Additional users from 40 colleges
Normal and seasonal traffic	10 times current peak traffic
Browser-based access	Browser and mobile app access
Internal university services	External API services
Basic availability	99.9% availability
Limited infrastructure	Higher scalability required

Importance of APIs

APIs become important because:

- Mobile applications need to access student information.
- Affiliated colleges need to retrieve student records.
- Payment systems need to communicate with university databases.
- Different applications must exchange information securely.

Multi-Tenant Security

Multi-tenancy means multiple colleges use the same cloud application while keeping their data separate.

Security measures:

- Give each college a unique tenant ID.
- Provide separate access permissions.
- Use role-based access control.
- Encrypt student information.
- Prevent one college from accessing another college's records.
- Maintain separate audit logs.

Conclusion: The new requirements need a more scalable, secure and highly available cloud architecture.

11.2 Evaluation of the Original Design (2 Marks)

1. Can the Original Design Handle 10x Traffic?

The original design uses three physical hosts and a combination of virtual machines and containers.

It is suitable for normal university workloads but may not be sufficient for 10 times the current peak traffic.

Therefore, additional application instances and managed cloud services are required.

2. Is Auto-Scaling Sufficient?

Auto-scaling helps increase application resources automatically.

However, auto-scaling alone cannot guarantee 99.9% availability if the entire application is hosted in one region.

The system should use multiple availability zones and a separate disaster recovery region.

3. Availability Calculation

Required availability = 99.9%.

Downtime = 100% − 99.9%

= 0.1%

Total hours in one year:

365 × 24 = 8760 hours

Allowed downtime:

8760 × 0.1 ÷ 100

= 8.76 hours

Maximum downtime: 8.76 hours per year.

4. Cost Evaluation

The increased traffic requires additional computing, database and network resources.

The original cloud design may become more expensive because more resources are consumed.

Using managed PaaS and serverless services can reduce infrastructure management costs.

5. Better Service Models

The revised solution should use:

- IaaS for legacy applications.
- PaaS for application hosting.
- SaaS for email.
- FaaS for small payment and notification operations.
- DBaaS for managed databases.
- DRaaS for disaster recovery.

Conclusion: The original design is a good starting point, but modifications are necessary to support the new requirements.

11.3 Adaptation or Redesign (2 Marks)

Decision: MODIFY the Original Design

The original hybrid cloud design should be modified instead of completely replaced.

Revised Cloud Architecture

The revised architecture includes the following components:

1. API Gateway: Manages requests from mobile applications and affiliated colleges.
2. Load Balancer: Distributes incoming requests across available servers.
3. Container Orchestration: Manages application containers and their resources.
4. Auto-Scaling: Automatically increases or decreases application resources.
5. Managed Database: Stores student records and examination information.
6. Serverless Functions: Handles payment notifications and other short operations.
7. Multi-Zone Deployment: Improves system availability.
8. Disaster Recovery Region: Maintains backup data and recovery services.

Changes in the Revised Architecture

Component	Original Design	Revised Design
Access	Web browser	Web and mobile applications

API	Basic REST APIs	Managed API Gateway
Application	Virtual machines and containers	Container orchestration
Database	Cloud database	Managed DBaaS with replication
Payment	Normal application	Serverless payment integration
Availability	Basic cloud deployment	Multi-zone deployment
Backup	Cloud backup	Cross-region DRaaS

Why These Changes Are Required

1. API Gateway manages requests from mobile applications and affiliated colleges.
2. Container orchestration manages application instances.
3. Auto-scaling supports increased traffic.
4. DBaaS reduces database maintenance.
5. Serverless functions handle payment notifications and other short operations.
6. Multi-zone deployment improves availability.
7. Disaster recovery protects university information during major failures.

Conclusion: Modifying the existing hybrid cloud design provides better performance, security and scalability without completely rebuilding the university's system.

11.4 Evidence-Based Defence (2 Marks)

1. Capacity Calculation

Assume each of the 12 existing servers represents one unit of CPU capacity.

Current peak utilization is approximately 95%.

Total current peak capacity:

$12 \times 0.95 = 11.4$

Expected peak after a 10x increase:

$11.4 \times 10 = 114$

Therefore, the university needs approximately 114 equivalent server-capacity units at peak.

If physical hosts are operated at a target utilization of 70%:

Required hosts = $114 \div 0.70$

$= 162.86$

Therefore:

163 equivalent physical hosts.

This is a normalized capacity illustration, not a final hardware purchase recommendation. The revised cloud design avoids buying 163 physical servers by using managed containers and elastic cloud resources. Actual sizing must be confirmed through load testing and CPU, RAM, storage and network measurements.

2. Availability Calculation

The target availability is 99.9%.

8760 × 0.001 = 8.76 hours.

The maximum permitted downtime is 8.76 hours per year.

To achieve this target, the university should use:

- Multiple availability zones.
- Load balancing.
- Database replication.
- Automatic health checks.
- Regular backups.
- A secondary disaster recovery region.

3. Cost Comparison

The following estimates are illustrative assumptions for comparing the original and revised designs.

All amounts in ₹ lakh for three years.

Expense	Original Cloud Design	Revised Cloud Design
Initial setup and migration	15	20
Monthly cloud expenses	3	5.5
Three-year cloud expenses	108	198
Additional annual management	0	6
Total	123	224

Calculation for the original design:

15 + (3 × 36) = ₹123 lakh.

Calculation for the revised design:

20 + (5.5 × 36) + 6 = ₹224 lakh.

The revised design costs more because it supports ten times the peak traffic, additional API services, database replication and disaster recovery.

However, elastic scaling can reduce costs during low-traffic periods.

4. Security Considerations

The revised system should provide:

- Multi-factor authentication.
- Role-based access control.
- Separate college tenant identities.
- Encrypted student records.
- Secure payment gateway integration.
- API rate limiting.
- Continuous monitoring and audit logs.

5. Trade-Offs

Advantage	Disadvantage
High scalability	Higher cloud expenses
Better availability	More complex architecture
Easy mobile integration	API security requirements
Reduced hardware maintenance	Vendor dependency
Disaster recovery	Additional backup costs

Final Decision

The university should modify its original hybrid cloud architecture by introducing an API Gateway, container orchestration, managed databases, serverless functions and multi-zone deployment.

This solution supports the mobile application, 40 affiliated colleges and increased traffic while maintaining the required availability target.

OVERALL ASSIGNMENT CONCLUSION

Cloud computing provides an effective solution for migrating the university's traditional infrastructure to a modern IT environment.

The proposed hybrid cloud architecture combines IaaS, PaaS, SaaS and XaaS services. Server virtualization improves hardware utilization, while web services allow different applications to communicate.

The revised architecture supports mobile applications, affiliated colleges, high traffic and secure online payments. Auto-scaling, load balancing, managed databases and disaster recovery improve system performance and availability.

Therefore, cloud migration is a suitable long-term solution for the university because it reduces infrastructure management, improves scalability and provides reliable services to students, teachers and staff.